シャチョツー 全社自動化設計書 — マーケ・SFA・CRM・CS + 学習ループ

> **ビジョン**: AIエージェントで会社が回る世界。人間は「判断が必要な場面」だけに集中する。
>
> **対象**: シャチョツー（自社プロダクト）を売り、支え、育てるための**社内AIエージェント群**。
> 社外SaaSではなく、自社の営業・CS・マーケ・バックオフィスをAIで全自動化する。
>
> **設計方針**:
> - 既存の3層マイクロエージェントアーキテクチャを最大活用
> - `shachotwo-マーケAI` / `shachotwo-契約AI` の使える部品を `shachotwo-app` に吸収統合
> - 学習フィードバックループを全レイヤーに組み込む

---

0. 全体ビジョン — AI×人間の役割分担

0.1 シャチョツーの事業成長ループ

```
  ブレイン導入            BPO#1 横展開          BPO追加横展開          バックオフィス        自社開発BPO
  (社長の脳デジタル化)     (業種マストニーズ)     (追加モジュール)       (経理/給与/勤怠)      (最終巻き取り)
  ──────────→ ──────────→ ──────────→ ──────────→ ──────────→
       ¥30K/月            ¥250K/月            +¥100K/個/月          +¥200K/月            別途見積

  【AIが自動】            【AIが自動】          【人間 + AI】          【人間 + AI】         【人間主導】
  リード獲得              オンボード            CSコンサルが提案       CSコンサルが設計       CSコンサルが主導
  提案書生成              初期設定              AI がデータで支援      AI が業務分析          AI はツール提供
  見積→契約               FAQ対応               顧客の利用データ→      効果シミュレーション
                                                拡張タイミング提案
```

0.2 AI vs 人間の役割マトリクス

フェーズ	AI がやること	人間（CSコンサルタント）がやること
認知→リード	企業リサーチ、アウトリーチ400件/日、LP生成、シグナル検知	—
リード→提案	スコアリング、業種判定、提案書AI生成、メール送付	スコア70未満の判断（Slack通知で確認）
提案→契約	見積自動計算、契約書生成、CloudSign署名、freee請求書	—
オンボード	テナント作成、ゲノム適用、ウェルカムメール、初期設定ガイド	初回キックオフMTG（Google Meet自動予約）
運用・CS	FAQ自動回答、ヘルススコア計算、SLAモニタリング	エスカレーション対応、複雑な問い合わせ
アップセル	利用データ分析→拡張タイミング検知→Slack通知	**商談・提案・クロージング（人間の仕事）**
解約防止	解約リスク検知→アラート→対策サジェスト	個別フォロー、リテンション交渉

> **人間が入るべき場所**: アップセル商談、複雑なカスタマイズ提案、解約交渉。
> これらは「関係性と信頼」が必要。AIは**データと準備**を提供する。

0.3 既存プロジェクト統合方針

プロジェクト	状態	統合方針
`shachotwo-マーケAI`	40%完成	**吸収**: research/enricher, signals/detector, scheduling/calendar_api, outreach/personalize, resume/generator を `shachotwo-app` に移植
`shachotwo-契約AI`	20%完成	**吸収**: contract/cloudsign, contract/estimate(WeasyPrint PDF), email_templates を `shachotwo-app` に移植
`shachotwo-app`	本体	上記を `workers/bpo/sales/` 配下に統合

統合後、元リポジトリはアーカイブ（削除はしない。参照用に残す）

---

1. 全体アーキテクチャ

```
┌──────────────────────────────────────────────────────────────────────────┐
│          シャチョツー 全社自動化エンジン（社内AIエージェント群）                │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │ Layer 0: Sales Manager（既存BPO Managerを拡張）                    │    │
│  │                                                                  │    │
│  │ EventListener / ScheduleWatcher / ConditionEvaluator             │    │
│  │ TaskRouter（パイプライン自動選択）                                  │    │
│  └──────────────────────────┬───────────────────────────────────────┘    │
│                             │                                            │
│  ┌──────────────────────────┴───────────────────────────────────────┐    │
│  │ Layer 1: パイプライン（10本）                                       │    │
│  │                                                                  │    │
│  │ 【マーケ】← shachotwo-マーケAI 吸収                                 │    │
│  │ ⓪ 企業リサーチ＆アウトリーチ（400件/日 自動）                          │    │
│  │                                                                  │    │
│  │ 【SFA】                                                            │    │
│  │ ① リードクオリフィケーション                                          │    │
│  │ ② 提案書AI生成・送付                                                 │    │
│  │ ③ 見積・契約・電子署名 ← shachotwo-契約AI 吸収                        │    │
│  │                                                                  │    │
│  │ 【CRM】                                                            │    │
│  │ ④ 顧客ライフサイクル管理                                              │    │
│  │ ⑤ 売上・要望管理                                                     │    │
│  │                                                                  │    │
│  │ 【CS】                                                             │    │
│  │ ⑥ サポート自動対応                                                   │    │
│  │                                                                  │    │
│  │ 【アップセル支援】                                                    │    │
│  │ ⑦ 拡張タイミング検知 & コンサルへのブリーフィング                       │    │
│  │                                                                  │    │
│  │ 【学習】                                                           │    │
│  │ ⑧ 受注/失注フィードバック学習                                         │    │
│  │ ⑨ CS対応品質フィードバック学習                                        │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │ Layer 2: マイクロエージェント（既存12種 + マーケAI由来4種）              │    │
│  │                                                                  │    │
│  │ 【既存】extractor / generator / rule_matcher / calculator          │    │
│  │         message / validator / saas_reader / saas_writer          │    │
│  │         ocr / compliance / diff / table_parser                   │    │
│  │                                                                  │    │
│  │ 【マーケAI由来】                                                     │    │
│  │ company_researcher  — 企業リサーチ＆ペイン推定（enricher.py）        │    │
│  │ signal_detector     — シグナル検知＆温度判定（detector.py）          │    │
│  │ pdf_generator       — PDF生成 WeasyPrint（契約AI由来）              │    │
```

```
│  │  calendar_booker     — Google Calendar空き枠＆予約（calendar_api）  │     │
│  └──────────────────────────────────────────────────────────────────┘     │
│                                                                            │
│  ┌──────────────────────────────────────────────────────────────────┐     │
│  │  コネクタ層                                                        │     │
│  │                                                                    │     │
│  │   【既存】 Slack / kintone / freee（+請求書拡張）                   │     │
│  │   【新規】 SendGrid / CloudSign / Google Calendar / gBizINFO       │     │
│  │   【マーケAI由来】 Playwright（フォーム自動送信）/ Google Sheets    │     │
│  └──────────────────────────────────────────────────────────────────┘     │
│                                                                            │
│  ┌──────────────────────────────────────────────────────────────────┐     │
│  │  学習フィードバックループ（全レイヤー横断）                          │     │
│  │                                                                    │     │
│  │  受注/失注 → スコアリングモデル更新 → 提案書テンプレ改善             │     │
│  │  CS評価 → FAQ自動更新 → AI回答品質向上                              │     │
│  │  解約理由 → ヘルススコア重み調整 → プロダクト改善                    │     │
│  │  アウトリーチ成果 → ターゲティング精度向上 → メール文改善            │     │
│  └──────────────────────────────────────────────────────────────────┘     │
└────────────────────────────────────────────────────────────────────────────┘
```

---

2. 全社ファネル全体フロー

```
  リサーチ        アウトリーチ      リード          提案          契約          オンボード       運用
アップセル
  ──────→ ──────→ ──────→ ──────→ ──────→ ──────→ ──────→ ──────→

  ┌────┐  ┌────┐  ┌────┐  ┌────┐  ┌────┐  ┌────┐  ┌────┐  ┌────┐
  │企業 │→│400件│→│スコア│→│提案書│→│見積 │→│テナ │→│CS   │→│拡張 │
  │発掘 │  │/日  │  │判定  │  │AI生成│  │契約 │  │ント │  │Bot  │  │提案 │
  │    │  │自動 │  │    │  │    │  │署名 │  │作成 │  │    │  │    │
  └────┘  └────┘  └────┘  └────┘  └────┘  └────┘  └────┘  └────┘
    │       │       │       │       │       │       │       │
   AI      AI      AI      AI      AI     AI+人    AI    人+AI
  100%    100%    100%    100%    100%    90%     95%    人主導

  ←──────────── 学習フィードバックループ ────────────────────────→
  アウトリーチ成果→スコア精度→提案品質→契約率→利用データ→解約予防
```

---

3. データモデル（新規テーブル）

3.1 ER図

```
companies（既存）
    │
    ├──< leads                    # リード管理
    │       │
    │       └──< lead_activities  # リードの行動ログ
    │
    ├──< opportunities            # 商談管理
    │       │
    │       ├──< proposals        # 提案書
    │       ├──< quotations       # 見積書
    │       └──< contracts        # 契約書
    │
    ├──< customers                # 顧客管理（契約後）
    │       │
    │       ├──< customer_health  # 顧客ヘルススコア
    │       ├──< revenue_records  # 売上記録
    │       └──< feature_requests # 要望管理
    │
    └──< support_tickets          # サポートチケット
            │
            └──< ticket_messages  # チケットメッセージ
```

3.2 テーブル定義

```
-- ============================================================
-- SFA テーブル
-- ============================================================

-- リード管理
CREATE TABLE leads (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),  -- 将来の社外展開用に残す（当面は固定
値）

    -- リード情報
    company_name TEXT NOT NULL,                -- 見込み企業名
    contact_name TEXT,                         -- 担当者名
    contact_email TEXT,                        -- メールアドレス
    contact_phone TEXT,                        -- 電話番号
    industry TEXT,                             -- 業種（16業種コード）
    employee_count INTEGER,                    -- 従業員数

    -- ソース・スコア
    source TEXT NOT NULL,                      -- 流入元: website / referral / event /
outbound
    source_detail TEXT,                        -- 詳細（フォームURL、紹介者名等）
    score INTEGER DEFAULT 0,                   -- AIスコア（0-100）
    score_reasons JSONB DEFAULT '[]',          -- スコア根拠

    -- ステータス
    status TEXT NOT NULL DEFAULT 'new',        -- new / contacted / qualified /
unqualified / nurturing
    assigned_to UUID REFERENCES users(id),     -- 担当者（NULLならAI自動対応）

    -- タイムスタンプ
    first_contact_at TIMESTAMPTZ,
    last_activity_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- リード行動ログ
CREATE TABLE lead_activities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    lead_id UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,

    activity_type TEXT NOT NULL,               -- page_view / form_submit / email_open /
email_click / meeting / call
    activity_data JSONB DEFAULT '{}',          -- 行動詳細
    channel TEXT,                              -- web / email / slack / phone

    created_at TIMESTAMPTZ DEFAULT NOW()
```

```
);

-- 商談管理
CREATE TABLE opportunities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    lead_id UUID REFERENCES leads(id),
    customer_id UUID REFERENCES customers(id),

    -- 商談情報
    title TEXT NOT NULL,                      -- 商談名
    target_company_name TEXT NOT NULL,        -- 対象企業名
    target_industry TEXT,                     -- 対象業種

    -- 金額・モジュール
    selected_modules JSONB NOT NULL DEFAULT '[]',  -- ["brain", "bpo_core",
"bpo_additional_1"]
    monthly_amount INTEGER NOT NULL DEFAULT 0,      -- 月額合計（円）
    annual_amount INTEGER GENERATED ALWAYS AS (monthly_amount * 12) STORED,

    -- パイプライン
    stage TEXT NOT NULL DEFAULT 'proposal',   -- proposal / quotation / negotiation /
contract / won / lost
    probability INTEGER DEFAULT 50,           -- 受注確度（%）
    expected_close_date DATE,                 -- 受注予定日
    lost_reason TEXT,                         -- 失注理由

    -- タイムスタンプ
    stage_changed_at TIMESTAMPTZ DEFAULT NOW(),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 提案書
CREATE TABLE proposals (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    opportunity_id UUID NOT NULL REFERENCES opportunities(id),

    version INTEGER NOT NULL DEFAULT 1,
    title TEXT NOT NULL,
    content JSONB NOT NULL,                   -- 提案書構造化データ
    pdf_storage_path TEXT,                    -- Supabase Storage パス

    -- 送付
    sent_at TIMESTAMPTZ,
    sent_to TEXT,                             -- 送付先メール
    opened_at TIMESTAMPTZ,                   -- 開封日時

    status TEXT NOT NULL DEFAULT 'draft',     -- draft / sent / viewed / accepted /
rejected
    created_at TIMESTAMPTZ DEFAULT NOW()
```

```
);

-- 見積書
CREATE TABLE quotations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    opportunity_id UUID NOT NULL REFERENCES opportunities(id),

    version INTEGER NOT NULL DEFAULT 1,
    quotation_number TEXT NOT NULL,           -- 見積番号（自動採番）

    -- 明細
    line_items JSONB NOT NULL,               -- [{module, unit_price, quantity,
subtotal}]
    subtotal INTEGER NOT NULL,               -- 小計
    tax INTEGER NOT NULL,                    -- 消費税
    total INTEGER NOT NULL,                  -- 合計
    valid_until DATE NOT NULL,               -- 有効期限

    -- 送付
    pdf_storage_path TEXT,
    sent_at TIMESTAMPTZ,
    sent_to TEXT,

    status TEXT NOT NULL DEFAULT 'draft',     -- draft / sent / accepted / rejected /
expired
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 契約書
CREATE TABLE contracts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    opportunity_id UUID NOT NULL REFERENCES opportunities(id),

    contract_number TEXT NOT NULL,            -- 契約番号
    contract_type TEXT NOT NULL DEFAULT 'subscription',  -- subscription / one_time

    -- 契約内容
    selected_modules JSONB NOT NULL,
    monthly_amount INTEGER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,                           -- NULL = 自動更新
    auto_renew BOOLEAN DEFAULT TRUE,

    -- 電子署名
    signing_service TEXT DEFAULT 'cloudsign',  -- cloudsign / docusign
    signing_request_id TEXT,                   -- 外部署名サービスID
    signed_at TIMESTAMPTZ,
    pdf_storage_path TEXT,

    status TEXT NOT NULL DEFAULT 'draft',      -- draft / sent / signed / active /
```

```
terminated
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================
-- CRM テーブル
-- ============================================================

-- 顧客管理（契約締結後にleadsから昇格）
CREATE TABLE customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    lead_id UUID REFERENCES leads(id),

    -- 企業情報
    customer_company_name TEXT NOT NULL,
    industry TEXT NOT NULL,
    employee_count INTEGER,

    -- 契約情報
    plan TEXT NOT NULL,                       -- brain / bpo_core / enterprise
    active_modules JSONB NOT NULL DEFAULT '[]',
    mrr INTEGER NOT NULL DEFAULT 0,           -- 月次経常収益

    -- ヘルス
    health_score INTEGER DEFAULT 100,         -- 0-100
    nps_score INTEGER,                        -- -100〜100
    last_nps_at TIMESTAMPTZ,

    -- ステータス
    status TEXT NOT NULL DEFAULT 'onboarding', -- onboarding / active / at_risk /
churned
    onboarded_at TIMESTAMPTZ,
    churned_at TIMESTAMPTZ,
    churn_reason TEXT,

    -- 担当
    cs_owner UUID REFERENCES users(id),       -- カスタマーサクセス担当

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 顧客ヘルススコア履歴
CREATE TABLE customer_health (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    customer_id UUID NOT NULL REFERENCES customers(id),

    score INTEGER NOT NULL,                   -- 0-100
    dimensions JSONB NOT NULL,                -- {usage: 80, engagement: 70, support:
90, nps: 60, expansion: 50}
```

```
    risk_factors JSONB DEFAULT '[]',          -- ["低ログイン頻度", "未回答NPS"]

    calculated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 売上記録
CREATE TABLE revenue_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    customer_id UUID NOT NULL REFERENCES customers(id),

    record_type TEXT NOT NULL,                -- mrr / expansion / contraction / churn
    amount INTEGER NOT NULL,                  -- 金額（円）
    modules JSONB,                            -- 対象モジュール
    effective_date DATE NOT NULL,

    -- freee連携
    freee_invoice_id INTEGER,                -- freee請求書ID
    payment_status TEXT DEFAULT 'pending',    -- pending / paid / overdue / failed

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 要望管理
CREATE TABLE feature_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    customer_id UUID NOT NULL REFERENCES customers(id),

    title TEXT NOT NULL,
    description TEXT NOT NULL,
    category TEXT,                             -- feature / improvement / integration /
bug
    priority TEXT DEFAULT 'medium',           -- low / medium / high / critical

    -- AIによる分類・集約
    ai_category JSONB,                        -- AI自動分類タグ
    similar_request_ids UUID[],               -- 類似要望のID
    vote_count INTEGER DEFAULT 1,             -- 同様要望のカウント

    status TEXT NOT NULL DEFAULT 'new',       -- new / reviewing / planned / in_progress
/ done / declined
    response TEXT,                            -- 回答内容
    responded_at TIMESTAMPTZ,

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================
-- CS テーブル
-- ============================================================
```

```
-- サポートチケット
CREATE TABLE support_tickets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    customer_id UUID NOT NULL REFERENCES customers(id),

    ticket_number TEXT NOT NULL,              -- チケット番号（自動採番）
    subject TEXT NOT NULL,

    -- 分類
    category TEXT NOT NULL,                   -- usage / billing / bug / feature /
account
    priority TEXT NOT NULL DEFAULT 'medium',  -- low / medium / high / urgent

    -- AI対応
    ai_handled BOOLEAN DEFAULT FALSE,         -- AI自動対応済みか
    ai_confidence FLOAT,                      -- AI回答の確信度
    ai_response TEXT,                         -- AI生成回答

    -- エスカレーション
    escalated BOOLEAN DEFAULT FALSE,
    escalated_to UUID REFERENCES users(id),
    escalation_reason TEXT,

    -- SLA
    sla_due_at TIMESTAMPTZ,                  -- SLA期限
    first_response_at TIMESTAMPTZ,
    resolved_at TIMESTAMPTZ,

    status TEXT NOT NULL DEFAULT 'open',      -- open / waiting / ai_responded /
escalated / resolved / closed
    satisfaction_score INTEGER,               -- 1-5

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- チケットメッセージ
CREATE TABLE ticket_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    ticket_id UUID NOT NULL REFERENCES support_tickets(id) ON DELETE CASCADE,

    sender_type TEXT NOT NULL,                -- customer / agent / ai
    sender_id UUID,                           -- users.id or NULL(AI)
    content TEXT NOT NULL,
    attachments JSONB DEFAULT '[]',

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================
```

```
-- RLS（全テーブル共通パターン）
-- ============================================================
-- ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
-- CREATE POLICY "tenant_isolation" ON leads
--   USING (company_id = current_setting('app.company_id')::UUID);
--  （全テーブルに同様に適用）

-- ============================================================
-- インデックス
-- ============================================================
CREATE INDEX idx_leads_status ON leads(company_id, status);
CREATE INDEX idx_leads_score ON leads(company_id, score DESC);
CREATE INDEX idx_opportunities_stage ON opportunities(company_id, stage);
CREATE INDEX idx_customers_health ON customers(company_id, health_score);
CREATE INDEX idx_customers_status ON customers(company_id, status);
CREATE INDEX idx_support_tickets_status ON support_tickets(company_id, status);
CREATE INDEX idx_support_tickets_sla ON support_tickets(sla_due_at) WHERE status !=
'closed';
CREATE INDEX idx_revenue_records_date ON revenue_records(company_id, effective_date);
CREATE INDEX idx_feature_requests_votes ON feature_requests(company_id, vote_count
DESC);
```

---

4. パイプライン詳細設計

4.0 マーケ パイプライン⓪ — 企業リサーチ＆アウトリーチ（マーケAI吸収）

> **吸収元**: `shachotwo-マーケAI/research/` , `outreach/` , `signals/` , `scheduling/` , `resume/`
> これらを `shachotwo-app/workers/bpo/sales/pipelines/` と `workers/micro/` に再配置する。

```
トリガー: ScheduleWatcher 毎日 8:00（自動）/ 手動トリガー
         ↓
┌──────────────────────────────────────────────────────────┐
│  outreach_pipeline.py                                     │
│                                                           │
│  Step 1: company_researcher（マーケAI由来・新micro）      │
│    └→ 求人サイトスクレイピング → 採用中企業リスト取得      │
│    └→ gBizINFO API で法人情報エンリッチ                   │
│       {company_name, industry, employee_count,            │
│        representative, address, corporate_number}         │
│    └→ HPスクレイピング → 事業概要・問い合わせフォーム取得 │
│                                                           │
│  Step 2: company_researcher → LLMペイン推定               │
│    └→ Gemini Flash: 企業情報 → ペインポイント推定         │
│       「従業員50名の建設会社 → 属人化・見積工数が課題」   │
│    └→ 業種判定 → 16業種コードにマッピング                 │
│    └→ 企業規模でトーン調整（小規模→親しみ/中堅→実績重視）│
│                                                           │
│  Step 3: generator（既存）→ カスタムLP/レジュメ生成       │
│    └→ 業種×ペインに合わせたLP内容を自動生成               │
│    └→ resume/templates/ の業種テンプレートを使用          │
│                                                           │
│  Step 4: message（既存）→ パーソナライズメール生成        │
│    └→ LLM: 企業情報+ペイン → 件名+本文を動的生成          │
│    └→ フォールバック: テンプレートメール                  │
│                                                           │
│  Step 5: Playwright / SendGrid → アウトリーチ実行         │
│    └→ 問い合わせフォームあり → Playwright自動送信         │
│    └→ フォームなし → SendGridメール送信                   │
│    └→ 1日400件ペース制御（特定電子メール法準拠）          │
│                                                           │
│  Step 6: signal_detector（マーケAI由来・新micro）         │
│    └→ LP閲覧・CTA クリック・資料DL をトラッキング         │
│    └→ 温度判定: hot / warm / cold                         │
│       ・hot（CTA3回以上）→ 即 lead_qualification へ       │
│       ・warm（LP閲覧あり）→ 3日後フォローアップメール     │
│       ・cold（反応なし）→ 7日後に別切り口で再アプローチ   │
│                                                           │
│  Step 7: calendar_booker（マーケAI由来・新micro）         │
│    └→ hot リードが商談希望 →                              │
│       Google Calendar API で空き枠提示                    │
│       → 自動でGoogle Meet URL付きの予約作成               │
│       → Slack に「商談予約入りました」通知                 │
│                                                           │
│  Step 8: saas_writer（既存）                              │
│    └→ leads テーブルに保存（source = "outreach"）         │
│    └→ lead_activities にアウトリーチログ記録              │
│    └→ Google Sheets に営業リスト同期（既存sheet/連携）    │
└──────────────────────────────────────────────────────────┘
```

日次アウトリーチ目標:

指標	目標	備考
リサーチ企業数	500社/日	求人サイト + gBizINFO
アウトリーチ数	400件/日	フォーム + メール
LP閲覧率	5-10%	20-40件/日
商談化率	2-3%	8-12件/日

4.1 SFA パイプライン① — リードクオリフィケーション

```
トリガー: フォーム送信 / Webhook / メール着信
          ↓
┌──────────────────────────────────────────────────────┐
│  lead_qualification_pipeline.py                      │
│                                                      │
│  Step 1: extractor（既存）                            │
│    └→ フォームデータから企業情報を構造化抽出            │
│       {company_name, industry, employee_count, need}  │
│                                                      │
│  Step 2: rule_matcher（既存）                         │
│    └→ リードスコアリング                               │
│        ・業種マッチ: 16業種該当 → +30pt               │
│        ・従業員数: 10-300名 → +20pt                   │
│        ・ニーズ緊急度: 即導入希望 → +30pt             │
│        ・予算感: BPOコア以上 → +20pt                  │
│                                                      │
│  Step 3: ConditionEvaluator（既存）                   │
│    └→ 自動/手動振り分け                                │
│        ・score ≥ 70: → 自動で提案書パイプラインへ       │
│        ・score 40-69: → Slack通知 → 営業判断待ち       │
│        ・score < 40: → ナーチャリング（メール自動配信）  │
│                                                      │
│  Step 4: saas_writer（既存）                          │
│    └→ leads テーブルに保存                             │
│    └→ Slack に通知送信                                 │
│                                                      │
│  Step 5: message（既存）                              │
│    └→ 初回お礼メール自動生成・送信                      │
└──────────────────────────────────────────────────────┘
```

スコアリングルール詳細:

ファクター	条件	スコア
業種マッチ	16対応業種に該当	+30
業種マッチ	対応業種外	+5
従業員規模	10-50名（コアターゲット）	+25
従業員規模	51-300名	+20
従業員規模	300名超 or 10名未満	+5
ニーズ緊急度	「すぐ導入したい」	+30
ニーズ緊急度	「検討中」	+15
ニーズ緊急度	「情報収集」	+5
予算感	BPOコア以上（月25万+）	+20
予算感	ブレインのみ（月3万）	+10
流入元	紹介	+15（ボーナス）
流入元	イベント	+10（ボーナス）

4.2 SFA パイプライン② — 提案書自動生成・送付

```
トリガー: リードスコア ≥ 70 / 営業が手動トリガー
         ↓
┌──────────────────────────────────────────────────────┐
│  proposal_generation_pipeline.py                      │
│                                                       │
│  Step 1: saas_reader（既存）                           │
│    └→ リード情報・企業情報を取得                          │
│                                                       │
│  Step 2: rule_matcher（既存）                          │
│    └→ 業種テンプレート選択                               │
│       genome/data/{industry}.json から業種特性取得       │
│                                                       │
│  Step 3: generator（既存）→ LLMで提案書生成              │
│    └→ 入力: リード情報 + 業種テンプレート + 料金体系      │
│    └→ 出力: 提案書JSON構造化データ                       │
│       {                                               │
│         "cover": "御社の課題に対するシャチョツー提案",     │
│         "pain_points": ["属人化", "情報断絶"],           │
│         "solution_map": [...],                        │
│         "modules": ["brain", "bpo_core"],             │
│         "pricing": {...},                             │
│         "roi_estimate": {...},                        │
│         "timeline": {...}                             │
│       }                                               │
│                                                       │
│  Step 4: generator（既存）→ PDF生成                    │
│    └→ テンプレートHTMLにデータ注入 → WeasyPrint PDF     │
│    └→ Supabase Storageにアップロード                    │
│                                                       │
│  Step 5: message（既存）→ 送付メール生成                 │
│    └→ 業種・課題に合わせたパーソナライズメール            │
│                                                       │
│  Step 6: EmailConnector（新規）→ メール送信             │
│    └→ 提案書PDF添付 + トラッキングリンク                 │
│    └→ 開封・クリック検知用Webhookを設定                  │
│                                                       │
│  Step 7: saas_writer（既存）                            │
│    └→ proposals テーブルに保存                          │
│    └→ opportunities ステージを "proposal" に更新        │
│    └→ Slack に送付完了通知                              │
└──────────────────────────────────────────────────────┘
```

提案書テンプレート構成:

1. 表紙（企業名・日付・担当者名）
2. 御社の課題認識（AIが業種＋リード情報から推定）
3. シャチョツーによる解決策
   - ブレイン: 社長の頭の中をデジタル化
   - BPO: 業種特化の業務自動化
4. 導入モジュール提案（業種に最適な組み合わせ）
5. 料金シミュレーション
6. ROI試算（業種平均データから推計）
7. 導入スケジュール
8. 事例（あれば同業種事例を自動挿入）
9. FAQ

4.3 SFA パイプライン③ — 見積書・契約書自動送付

```
トリガー: 提案書開封 + 質問回答完了 / 営業が手動トリガー
          ↓
┌──────────────────────────────────────────────────────┐
│  quotation_contract_pipeline.py                      │
│                                                      │
│  ── Phase A: 見積書 ──                               │
│                                                      │
│  Step 1: calculator（既存）                          │
│    └→ 選択モジュールから見積金額を自動計算           │
│        ・ブレイン: ¥30,000/月                        │
│        ・BPOコア: ¥250,000/月                        │
│        ・追加モジュール: ¥100,000/個/月              │
│        ・年払い割引: 10%                             │
│        ・紹介割引: 初月無料                          │
│                                                      │
│  Step 2: generator（既存）→ 見積書PDF生成            │
│    └→ 見積番号自動採番（QT-YYYYMM-XXXX）            │
│    └→ 有効期限: 発行日+30日                          │
│                                                      │
│  Step 3: EmailConnector → 見積書送付                 │
│                                                      │
│  Step 4: validator（既存）→ 承認確認待ち             │
│    └→ 見積承認 → Phase B へ                          │
│    └→ 修正要求 → Step 1 に戻る                       │
│    └→ 30日経過 → フォローアップメール自動送信        │
│                                                      │
│  ── Phase B: 契約書 ──                               │
│                                                      │
│  Step 5: generator（既存）→ 契約書生成               │
│    └→ 契約テンプレート + 見積内容からカスタマイズ    │
│    └→ 契約期間・自動更新・SLA条項を含む              │
│                                                      │
│  Step 6: CloudSignConnector（新規）→ 電子署名送信    │
│    └→ CloudSign APIで署名依頼を作成                  │
│    └→ Webhook で署名完了を検知                       │
│                                                      │
│  Step 7: saas_writer（既存）                         │
│    └→ contracts テーブルに保存                       │
│    └→ opportunities ステージを "won" に更新          │
│    └→ 自動で customers テーブルにレコード作成        │
│    └→ テナント自動プロビジョニング開始               │
│    └→ Slack に受注祝い通知                           │
│                                                      │
│  Step 8: 請求書発行                                  │
│    └→ freeeConnector（既存）で請求書を自動作成       │
│    └→ EmailConnector で請求書PDF送付                 │
│    └→ 入金確認は freee Webhook or 手動マーク         │
└──────────────────────────────────────────────────────┘
```

4.4 CRM パイプライン④ — 顧客ライフサイクル管理

```
トリガー: 契約締結 / 日次スケジュール / イベント
          ↓
┌──────────────────────────────────────────────────────┐
│  customer_lifecycle_pipeline.py                      │
│                                                      │
│  ── オンボーディング自動化 ──                          │
│                                                      │
│  Step 1: 顧客アカウントセットアップ                     │
│    └→ customers テーブルのステータスを onboarding に    │
│    └→ シャチョツー本体側で顧客企業のテナント作成        │
│    └→ 業種ゲノム（genome/data/{industry}.json）を適用  │
│    └→ 顧客の管理者ユーザーに招待メール送信             │
│                                                      │
│  Step 2: message（既存）→ ウェルカムメール送信         │
│    └→ ログインURL + 初期設定ガイド + 担当者紹介        │
│                                                      │
│  Step 3: ScheduleWatcher（既存）→ オンボードフォロー    │
│    └→ Day 1: ウェルカムメール                          │
│    └→ Day 3: 「初期設定は完了しましたか？」             │
│    └→ Day 7: 「最初のナレッジを登録しましょう」         │
│    └→ Day 14: 「BPO機能を試してみましょう」             │
│    └→ Day 30: NPS調査送信                              │
│                                                      │
│  ── ヘルススコア計算（日次） ──                        │
│                                                      │
│  Step 4: saas_reader（既存）→ 利用データ収集           │
│    └→ ログイン頻度 / 機能利用率 / Q&A回数 / BPO実行数  │
│                                                      │
│  Step 5: calculator（既存）→ ヘルススコア算出          │
│    └→ 5次元スコア:                                     │
│        ・利用度（WAU, DAU）        : 30%               │
│        ・エンゲージメント（機能幅）: 25%               │
│        ・サポート（チケット頻度）   : 15%               │
│        ・NPS                       : 15%               │
│        ・拡張可能性（未使用モジュール): 15%            │
│                                                      │
│  Step 6: rule_matcher（既存）→ アラート判定            │
│    └→ score < 40: 「解約リスク」→ CS担当にSlack緊急通知 │
│    └→ score 40-60: 「注意」→ 週次レポートに含める      │
│    └→ score > 80 + 未使用モジュールあり: 「拡張提案」  │
│                                                      │
│  Step 7: message（既存）→ 自動アクション               │
│    └→ 解約リスク: CS担当にアラート + 対策サジェスト    │
│    └→ 拡張提案: 追加モジュール提案メール自動生成       │
└──────────────────────────────────────────────────────┘
```

4.5 CRM パイプライン⑤ — 売上・要望管理

```
┌──────────────────────────────────────────────────────┐
│  revenue_request_pipeline.py                          │
│                                                       │
│  ── 売上管理（月次自動） ──                            │
│                                                       │
│  Step 1: freeeConnector → 請求・入金データ取得         │
│    └→ freee APIで入金ステータスを確認                  │
│    └→ revenue_records に自動記録                       │
│                                                       │
│  Step 2: calculator（既存）→ MRR/ARR/Churn率算出       │
│    └→ MRR = Σ(active_customers.mrr)                   │
│    └→ Net Revenue Retention (NRR)                     │
│    └→ 月次チャーン率                                   │
│    └→ 拡張MRR / 縮小MRR / 新規MRR                      │
│                                                       │
│  Step 3: generator（既存）→ 月次レポート自動生成       │
│    └→ ダッシュボード用JSONデータ                       │
│    └→ Slack月次サマリー投稿                            │
│                                                       │
│  ── 要望管理（随時） ──                                │
│                                                       │
│  Step 4: extractor（既存）→ 要望を構造化               │
│    └→ サポートチケット・Slack・メールから要望を自動抽出  │
│    └→ カテゴリ分類 + 類似要望のグルーピング            │
│                                                       │
│  Step 5: rule_matcher（既存）→ 優先度判定              │
│    └→ 要望頻度 × 顧客MRR × 解約リスク → 優先スコア     │
│    └→ 上位要望をプロダクトバックログに自動追加         │
│                                                       │
│  Step 6: message（既存）→ 顧客への回答                 │
│    └→ 「ご要望を承りました。検討状況をお知らせします」  │
│    └→ ステータス変更時に自動通知                       │
└──────────────────────────────────────────────────────┘
```

4.6 CS パイプライン⑥ — サポート自動対応

```
トリガー: メール着信 / チャット / フォーム
          ↓
┌──────────────────────────────────────────────────────┐
│  support_auto_response_pipeline.py                    │
│                                                       │
│  Step 1: extractor（既存）→ チケット情報構造化        │
│    └→ 件名・本文からカテゴリ・緊急度を自動判定         │
│    └→ 顧客情報を自動紐付け                            │
│                                                       │
│  Step 2: saas_reader（既存）→ コンテキスト収集        │
│    └→ 顧客の契約情報・利用状況・過去チケットを取得    │
│    └→ brain/knowledge/ でFAQ・ナレッジベースを検索    │
│                                                       │
│  Step 3: generator（既存）→ AI回答生成                │
│    └→ LLM: 問い合わせ + コンテキスト → 回答ドラフト   │
│    └→ confidence score を算出                         │
│                                                       │
│  Step 4: ConditionEvaluator（既存）→ 対応振り分け     │
│    ┌──────────────────────────────────────┐          │
│    │ confidence ≥ 0.85 → AI自動回答送信    │          │
│    │ confidence 0.5-0.85 → AIドラフト + 人間レビュー│  │
│    │ confidence < 0.5 → 即エスカレーション  │          │
│    │ category = billing → 経理チームにルーティング │  │
│    │ priority = urgent → 即Slackアラート    │          │
│    └──────────────────────────────────────┘          │
│                                                       │
│  Step 5: message（既存）→ 回答送信                    │
│    └→ メール or チャットで回答                        │
│    └→ 「この回答は役に立ちましたか？」フィードバックリンク │
│                                                       │
│  Step 6: validator（既存）→ SLA監視                   │
│    └→ 初回応答SLA: 1時間（AI自動対応でほぼ即時達成）  │
│    └→ 解決SLA: 24時間（urgent: 4時間）                │
│    └→ SLA違反予測 → エスカレーション                  │
│                                                       │
│  Step 7: saas_writer（既存）                          │
│    └→ support_tickets + ticket_messages に保存        │
│    └→ 解決時に satisfaction_score を収集              │
│    └→ customer_health に反映                          │
└──────────────────────────────────────────────────────┘
```

4.7 アップセル支援パイプライン⑦ — コンサルへのブリーフィング

> **人間のCSコンサルタントが主役。**AIはデータ収集・分析・タイミング検知・資料準備を行い、
> コンサルが「何を、いつ、どう提案すべきか」を判断できる状態を作る。

```
トリガー: 日次ヘルススコア計算後 / 利用マイルストーン到達
          ↓
┌──────────────────────────────────────────────────────────┐
│  upsell_briefing_pipeline.py                              │
│                                                           │
│  Step 1: saas_reader（既存）→ 顧客利用データ収集          │
│    └→ 過去30日の機能利用率・Q&A回数・BPO実行回数          │
│    └→ 現在の契約モジュール vs 未使用モジュール一覧        │
│    └→ 業種ゲノムの推奨モジュールマップ                    │
│                                                           │
│  Step 2: rule_matcher（既存）→ 拡張タイミング判定         │
│    ┌──────────────────────────────────────────┐          │
│    │ BPOコア利用率 ≥ 80% + 未使用モジュールあり          │
│    │   → 「追加モジュール提案タイミング」                 │
│    │                                                      │
│    │ ブレインのみ契約 + Q&A 週10回以上                    │
│    │   → 「BPOコアアップグレード提案」                    │
│    │                                                      │
│    │ health_score ≥ 80 + 契約6ヶ月経過                    │
│    │   → 「バックオフィスBPO提案」                        │
│    │                                                      │
│    │ 全BPOモジュール利用中 + カスタム要望3件以上          │
│    │   → 「自社開発BPO提案（コンサル必須）」              │
│    └──────────────────────────────────────────┘          │
│                                                           │
│  Step 3: generator（既存）→ コンサル用ブリーフィング      │
│    └→ AI生成ドキュメント:                                 │
│        ・顧客プロファイル要約                             │
│        ・現在の利用状況サマリー                           │
│        ・推奨アクション（根拠付き）                       │
│        ・過去のやり取り履歴                               │
│        ・想定質問と回答案                                 │
│        ・見積シミュレーション                             │
│                                                           │
│  Step 4: message（既存）→ Slack通知                       │
│    └→ #sales-upsell チャンネルに投稿:                     │
│        「🔔 [建設A社] BPOコア利用率85%到達。              │
│        追加モジュール"安全書類AI"の提案タイミングです。   │
│        ブリーフィング: [リンク]」                         │
│                                                           │
│  Step 5: calendar_booker（新micro）                       │
│    └→ コンサルのカレンダーに「提案準備」ブロック自動追加  │
│    └→ 顧客との商談候補日を3枠提示                         │
└──────────────────────────────────────────────────────────┘
```

4.8 学習パイプライン⑧ — 受注/失注フィードバック

> **これが最重要**。このループがないとAIは永遠に同じ精度のまま。
> 全てのアウトリーチ・提案・商談の結果を学習データとして蓄積し、
> スコアリング・提案書・メール文を自動改善する。

```
トリガー: 商談ステージが won / lost に変更された時
         ↓
┌──────────────────────────────────────────────────────┐
│  win_loss_feedback_pipeline.py                        │
│                                                       │
│  ── 受注時 ──                                         │
│                                                       │
│  Step 1: extractor（既存）→ 受注パターン抽出          │
│    └→ 何が刺さったか:                                 │
│        ・業種 / 従業員規模 / ペインポイント            │
│        ・提案書のどのセクションが響いたか              │
│        ・リードソース（アウトリーチ / インバウンド）    │
│        ・セールスサイクル日数                          │
│        ・選択されたモジュール                          │
│                                                       │
│  Step 2: rule_matcher 更新                            │
│    └→ リードスコアリングルールの重み調整:              │
│       受注した業種+規模の組み合わせ → スコア加点UP     │
│       例: 建設業×50名規模の受注率が高い                │
│           → 同条件のスコアボーナスを +5pt に調整       │
│                                                       │
│  Step 3: 提案書テンプレート改善                        │
│    └→ 受注案件の提案書を「成功テンプレート」として保存  │
│    └→ 業種別の勝ちパターンDB（win_patterns テーブル）  │
│    └→ 次回の提案書AI生成時にFew-shotとして参照         │
│                                                       │
│  ── 失注時 ──                                         │
│                                                       │
│  Step 4: message（既存）→ 失注理由ヒアリングメール     │
│    └→ 自動送信:「ご検討ありがとうございました。        │
│       今後の改善のため、お見送りの理由をお聞かせください」 │
│    └→ 選択肢: 価格/機能不足/他社選定/時期尚早/不要     │
│                                                       │
│  Step 5: extractor → 失注パターン分析                  │
│    └→ 失注理由を構造化 → lost_reasons テーブルに蓄積   │
│    └→ 月次集計: 失注理由ランキング                     │
│    └→ 価格が理由の失注が多い → 料金体系見直しアラート  │
│    └→ 機能不足が理由 → feature_requests に自動登録     │
│                                                       │
│  ── アウトリーチ成果学習（日次） ──                    │
│                                                       │
│  Step 6: アウトリーチPDCA自動最適化                    │
│    └→ 業種別の反応率（LP閲覧率・商談化率）を集計       │
│    └→ 反応率が高い業種 → アウトリーチ優先度UP          │
│    └→ 反応率が低い業種 → メール文A/Bテスト実行         │
│    └→ 件名・本文のバリエーションを自動生成             │
│    └→ 勝ちパターンを自動採用                           │
└──────────────────────────────────────────────────────┘
```

学習データの流れ:

```
【入力】                          【学習】                          【出力改善】

アウトリーチ400件/日 ──→ 反応率集計 ──→ ターゲティング精度UP
                                                    メール文バリエーション改善

リードスコア判定 ──→ 受注/失注と突合 ──→ スコアリング重み自動調整
                                                    高精度な自動/手動振り分け

提案書送付 ──→ 開封率・受注率 ──→ 業種別テンプレート改善
                                              成功パターンのFew-shot蓄積

CS回答 ──→ CSAT・解決率 ──→ FAQ自動更新
                                 AI回答品質向上
                                 エスカレ判断精度UP

ヘルススコア ──→ 実際の解約と突合 ──→ スコア次元の重み調整
                                                 解約予兆の早期検知精度UP

アップセル提案 ──→ 成約/不成約 ──→ 拡張タイミング判定精度UP
                                              ブリーフィング品質改善
```

4.9 学習パイプライン⑨ — CS対応品質フィードバック

```
トリガー: チケットクローズ時 / 月次バッチ
          ↓
┌──────────────────────────────────────────────────────┐
│  cs_feedback_pipeline.py                              │
│                                                       │
│  Step 1: saas_reader → クローズ済みチケット収集       │
│    └→ AI回答 vs 人間回答 の比率                       │
│    └→ CSAT スコア分布                                 │
│    └→ エスカレーション理由の分類                      │
│                                                       │
│  Step 2: extractor → パターン分析                     │
│    └→ CSAT ≥ 4 の AI回答 → 「良い回答」として保存     │
│    └→ CSAT ≤ 2 の AI回答 → 「改善必要」としてフラグ   │
│    └→ 人間がAI回答を修正したケース → 修正パターン抽出 │
│                                                       │
│  Step 3: knowledge/qa 自動更新                        │
│    └→ 新しいFAQパターンを knowledge_items に追加      │
│    └→ 既存FAQの回答を改善版で更新                     │
│    └→ エンベディング再計算                            │
│                                                       │
│  Step 4: confidence閾値の自動調整                     │
│    └→ AI自動回答のCSAT平均が4.0未満 →                 │
│       confidence閾値を 0.85 → 0.90 に引き上げ         │
│    └→ AI自動回答のCSAT平均が4.5以上 →                 │
│       confidence閾値を 0.85 → 0.80 に引き下げ         │
│       （自動対応率UP）                                │
│                                                       │
│  Step 5: 月次レポート生成                             │
│    └→ AI対応率推移 / CSAT推移 / よくある質問TOP10     │
│    └→ 「今月のAI改善提案」をSlack投稿                 │
└──────────────────────────────────────────────────────┘
```

4.10 学習フィードバックループ — データベーステーブル追加

```
-- 受注/失注パターン（学習データ）
CREATE TABLE win_loss_patterns (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    opportunity_id UUID NOT NULL REFERENCES opportunities(id),

    outcome TEXT NOT NULL,                    -- won / lost
    industry TEXT,                            -- 業種
    employee_range TEXT,                      -- 規模帯
    lead_source TEXT,                         -- リードソース
    sales_cycle_days INTEGER,                 -- セールスサイクル日数
    selected_modules JSONB,                   -- 選択モジュール
    lost_reason TEXT,                         -- 失注理由（lostの場合）
    win_factors JSONB,                        -- 受注要因（wonの場合）
    proposal_version_id UUID,                 -- 使用した提案書

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- アウトリーチ成果（PDCA用）
CREATE TABLE outreach_performance (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),

    period DATE NOT NULL,                     -- 集計期間（日次）
    industry TEXT NOT NULL,                   -- 業種

    -- 漏斗指標
    researched_count INTEGER DEFAULT 0,       -- リサーチ数
    outreached_count INTEGER DEFAULT 0,       -- アウトリーチ数
    lp_viewed_count INTEGER DEFAULT 0,        -- LP閲覧数
    lead_converted_count INTEGER DEFAULT 0,   -- リード化数
    meeting_booked_count INTEGER DEFAULT 0,   -- 商談予約数

    -- メールA/Bテスト
    email_variant TEXT,                       -- メールバリアント名
    open_rate FLOAT,                          -- 開封率
    click_rate FLOAT,                         -- クリック率

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- CS学習データ
CREATE TABLE cs_feedback (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    ticket_id UUID NOT NULL REFERENCES support_tickets(id),

    ai_response TEXT,                         -- AI生成回答
```

```
    human_correction TEXT,                    -- 人間による修正（あれば）
    csat_score INTEGER,                       -- 顧客満足度
    was_escalated BOOLEAN DEFAULT FALSE,

    -- 学習判定
    quality_label TEXT,                       -- good / needs_improvement / bad
    improvement_applied BOOLEAN DEFAULT FALSE, -- FAQに反映済みか

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- スコアリングモデルバージョン管理
CREATE TABLE scoring_model_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),

    model_type TEXT NOT NULL,                 -- lead_score / health_score /
upsell_timing
    version INTEGER NOT NULL,
    weights JSONB NOT NULL,                   -- スコアリング重み
    performance_metrics JSONB,                -- 精度指標（適合率・再現率等）

    active BOOLEAN DEFAULT FALSE,             -- 現在使用中か
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_win_loss_industry ON win_loss_patterns(industry, outcome);
CREATE INDEX idx_outreach_perf ON outreach_performance(period, industry);
CREATE INDEX idx_cs_feedback_quality ON cs_feedback(quality_label);
```

---

5. 新規コネクタ設計

5.1 EmailConnector（SMTP/IMAP）

```
class EmailConnector(BaseConnector):
    """メール送受信コネクタ"""

    async def send_email(self, to: str, subject: str, body: str,
                         attachments: list[str] = None) -> dict:
        """メール送信（提案書・見積書・契約書の送付に使用）"""

    async def fetch_inbound(self, since: datetime = None) -> list[dict]:
        """受信メール取得（リード獲得・サポート問い合わせに使用）"""

    async def setup_tracking(self, message_id: str) -> str:
        """開封・クリックトラッキング設定"""

    async def health_check(self) -> bool:
        """接続確認"""
```

実装方針:

- 送信: SendGrid API（トラッキング機能内蔵）
- 受信: IMAP polling or SendGrid Inbound Parse Webhook
- トラッキング: SendGrid Event Webhook → lead_activities に記録

5.2 CloudSignConnector

```
class CloudSignConnector(BaseConnector):
    """電子署名コネクタ（CloudSign API v2）"""

    async def create_document(self, pdf_path: str,
                              signers: list[dict]) -> str:
        """署名依頼作成 → signing_request_id を返す"""

    async def check_status(self, request_id: str) -> str:
        """署名ステータス確認"""

    async def download_signed(self, request_id: str) -> bytes:
        """署名済みPDFダウンロード"""

    async def handle_webhook(self, payload: dict) -> dict:
        """署名完了Webhook処理"""
```

5.3 freee請求書連携（既存freeeConnectorを拡張）

> 既存の `workers/connector/freee.py` を拡張して請求書発行・入金管理を追加する。
> Stripeサブスク管理は不要（社内ツールのため）。請求書はfreee経由で発行・管理。

```
# 既存 freee.py に以下メソッドを追加
class FreeeConnector(BaseConnector):
    # ... 既存メソッド（会計連携）...

    # ── 請求書管理（新規追加） ──

    async def create_invoice(self, partner_name: str,
                             line_items: list[dict],
                             due_date: date) -> dict:
        """freee請求書作成（契約締結時に自動発行）"""

    async def send_invoice(self, invoice_id: int,
                           to_email: str) -> dict:
        """請求書メール送付"""

    async def check_payment_status(self, invoice_id: int) -> str:
        """入金ステータス確認（未入金/入金済み/期限超過）"""

    async def list_overdue_invoices(self) -> list[dict]:
        """未入金請求書一覧（催促メール自動送信のトリガーに使用）"""
```

実装方針:

- freee会計API v1 の請求書エンドポイントを使用
- 契約締結 → freee請求書自動作成 → メール送付 → 入金確認
- 月次: 未入金チェック → 催促メール自動送信（message micro-agent）
- 入金確認: freee Webhook or 日次ポーリング → revenue_records に記録

5.4 IntercomConnector（オプション）

```
class IntercomConnector(BaseConnector):
    """カスタマーサポートチャットコネクタ"""

    async def create_conversation(self, user_id: str,
                                  body: str) -> dict:
        """会話作成"""

    async def reply(self, conversation_id: str, body: str) -> dict:
        """AI回答送信"""

    async def handle_webhook(self, payload: dict) -> dict:
        """新規メッセージWebhook処理"""
```

5.5 マーケAI由来コネクタ（吸収統合）

```
# --- gBizINFO コネクタ（マーケAI/research/gbizinfo.py 由来）---
class GBizInfoConnector(BaseConnector):
    """日本法人情報API（経産省 gBizINFO）"""

    async def search_company(self, name: str) -> dict:
        """企業名で法人情報検索（法人番号・代表者・従業員数・所在地）"""

    async def get_by_corporate_number(self, number: str) -> dict:
        """法人番号で詳細取得"""

# --- Playwright フォーム自動送信（マーケAI/outreach/form_sender.py 由来）---
class PlaywrightFormSender:
    """ヘッドレスブラウザによるフォーム自動送信"""

    async def detect_form_fields(self, url: str) -> list[dict]:
        """フォームフィールド自動検出"""

    async def submit_form(self, url: str, data: dict) -> bool:
        """フォーム自動入力・送信（人間的な遅延付き）"""

# --- Google Calendar コネクタ（マーケAI/scheduling/calendar_api.py 由来）---
class GoogleCalendarConnector(BaseConnector):
    """Google Calendar API 連携"""

    async def get_available_slots(self, days: int = 5) -> list[dict]:
        """空き時間枠を取得"""

    async def create_meeting(self, slot: dict, attendee_email: str,
                             title: str) -> dict:
        """Google Meet付きの予定を作成"""

# --- Google Sheets コネクタ（マーケAI/sheet/ 由来）---
class GoogleSheetsConnector(BaseConnector):
    """Google Sheets 読み書き（営業リスト管理）"""

    async def read_rows(self, range: str) -> list[dict]:
        """シートからデータ読込"""

    async def append_row(self, data: dict) -> None:
        """シートにデータ追記"""
```

実装方針:

- 全て `workers/connector/` に BaseConnector パターンで統合
- gBizINFO: 無料API、即時承認。マーケAI の実装をそのまま移植
- Playwright: フォーム送信専用。Chromium ヘッドレスモード
- Google Calendar/Sheets: OAuth 2.0、既存 Google credentials を共有

6. API設計（FastAPI ルーター）

6.0 routers/marketing.py — マーケエンドポイント

```
POST   /api/marketing/outreach/run          # アウトリーチ手動実行
GET    /api/marketing/outreach/status        # 本日のアウトリーチ状況
GET    /api/marketing/outreach/performance   # 業種別パフォーマンス

GET    /api/marketing/research/companies     # リサーチ済み企業一覧
POST   /api/marketing/research/enrich        # 企業エンリッチ手動実行

GET    /api/marketing/signals                # シグナル一覧（hot/warm/cold）
GET    /api/marketing/ab-tests               # A/Bテスト結果
```

6.1 routers/sales.py — SFAエンドポイント

```
POST   /api/sales/leads                    # リード登録（Webhook/フォーム）
GET    /api/sales/leads                    # リード一覧（フィルタ・ソート）
GET    /api/sales/leads/{id}               # リード詳細
PATCH  /api/sales/leads/{id}               # リード更新（ステータス変更等）
POST   /api/sales/leads/{id}/qualify       # リードクオリフィケーション実行

POST   /api/sales/opportunities            # 商談作成
GET    /api/sales/opportunities             # 商談一覧（パイプラインボード用）
PATCH  /api/sales/opportunities/{id}       # 商談更新（ステージ移動）
GET    /api/sales/opportunities/forecast    # 売上予測

POST   /api/sales/proposals/{opp_id}/generate   # 提案書AI生成
POST   /api/sales/proposals/{id}/send           # 提案書送付
GET    /api/sales/proposals/{id}/tracking       # 開封・閲覧状況

POST   /api/sales/quotations/{opp_id}/generate  # 見積書自動生成
POST   /api/sales/quotations/{id}/send          # 見積書送付
PATCH  /api/sales/quotations/{id}/approve       # 見積承認

POST   /api/sales/contracts/{opp_id}/generate   # 契約書生成
POST   /api/sales/contracts/{id}/sign           # 電子署名送信
POST   /api/sales/contracts/{id}/webhook        # CloudSign Webhook受信
```

6.2 routers/crm.py — CRMエンドポイント

```
GET    /api/crm/customers                  # 顧客一覧
GET    /api/crm/customers/{id}             # 顧客詳細（360度ビュー）
GET    /api/crm/customers/{id}/health      # ヘルススコア履歴
GET    /api/crm/customers/{id}/timeline    # アクティビティタイムライン
PATCH  /api/crm/customers/{id}             # 顧客情報更新

GET    /api/crm/revenue/summary            # MRR/ARR/NRRサマリー
GET    /api/crm/revenue/monthly            # 月次売上推移
GET    /api/crm/revenue/cohort             # コホート分析

GET    /api/crm/requests                   # 要望一覧
POST   /api/crm/requests                   # 要望登録
PATCH  /api/crm/requests/{id}              # 要望ステータス更新
GET    /api/crm/requests/ranking           # 要望ランキング（優先度順）
```

6.3 routers/support.py — CSエンドポイント

```
POST   /api/support/tickets                # チケット作成
GET    /api/support/tickets                # チケット一覧
GET    /api/support/tickets/{id}           # チケット詳細
PATCH  /api/support/tickets/{id}           # チケット更新
POST   /api/support/tickets/{id}/messages  # メッセージ追加
POST   /api/support/tickets/{id}/escalate  # エスカレーション

POST   /api/support/inbound/email          # 受信メールWebhook
POST   /api/support/inbound/chat           # チャットWebhook

GET    /api/support/metrics                # CS KPI（CSAT, FRT, Resolution Time）
```

6.4 Webhook受信エンドポイント

```
POST   /api/webhooks/sendgrid              # メール開封・クリック・受信
POST   /api/webhooks/cloudsign             # 電子署名完了
POST   /api/webhooks/freee                 # freee請求書入金通知
POST   /api/webhooks/intercom              # チャットメッセージ（オプション）
```

6.5 routers/upsell.py — アップセル支援エンドポイント

```
GET    /api/upsell/opportunities           # アップセル候補一覧
GET    /api/upsell/briefing/{customer_id}  # コンサル用ブリーフィング取得
POST   /api/upsell/briefing/{customer_id}/generate  # ブリーフィング再生成
```

6.6 routers/learning.py — 学習フィードバックエンドポイント

```
POST   /api/learning/win-loss/{opp_id}       # 受注/失注フィードバック登録
GET    /api/learning/win-loss/patterns        # 受注/失注パターン分析
GET    /api/learning/scoring/current          # 現在のスコアリングモデル
POST   /api/learning/scoring/retrain          # スコアリングモデル再学習
GET    /api/learning/outreach/performance     # アウトリーチPDCA指標
GET    /api/learning/cs/feedback-summary      # CS品質フィードバックサマリー
```

---

7. フロントエンド画面設計

7.0 マーケ画面

画面	パス	主要機能
アウトリーチダッシュボード	`/marketing/outreach`	本日の送信数・反応数・ホットリード。業種別パフォーマンス。ワンクリック実行
リサーチ企業一覧	`/marketing/research`	エンリッチ済み企業リスト。ペインポイント・温度表示
A/Bテスト	`/marketing/ab-tests`	メール件名・本文のバリアント別成果比較

7.1 SFA画面

画面	パス	主要機能
リードボード	`/sales/leads`	カンバンボード（new/contacted/qualified/nurturing）。スコア表示。ワンクリック提案書生成
商談パイプライン	`/sales/pipeline`	カンバンボード（proposal→quotation→negotiation→contract→won）。金額・確度表示
提案書管理	`/sales/proposals`	提案書一覧。生成・送付・開封状況。PDF プレビュー
見積・契約	`/sales/contracts`	見積書一覧 + 契約書一覧。ステータス管理。署名状況
売上予測	`/sales/forecast`	パイプライン金額 × 確度の加重予測。月別グラフ

7.2 CRM画面

画面	パス	主要機能
顧客一覧	`/crm/customers`	ヘルススコア順。ステータスバッジ。MRR表示
顧客360度	`/crm/customers/{id}`	契約情報・利用状況・ヘルス推移・チケット履歴・要望一覧を1画面に
売上ダッシュボード	`/crm/revenue`	MRR推移・NRR・チャーン率・コホート分析
要望ボード	`/crm/requests`	要望ランキング。投票数・MRRインパクト。ステータス管理

7.3 CS画面

画面	パス	主要機能
チケット一覧	`/support/tickets`	SLA残時間表示。AI対応済み/人間対応中フィルタ。優先度ソート
チケット詳細	`/support/tickets/{id}`	メッセージスレッド。AI回答プレビュー。エスカレーションボタン
CS KPI	`/support/metrics`	CSAT・平均初回応答時間・AI対応率・SLA達成率

7.4 アップセル・学習画面

画面	パス	主要機能
アップセル候補	`/upsell`	拡張タイミング到達した顧客一覧。ブリーフィングリンク。商談予約ボタン
コンサルブリーフィング	`/upsell/{customer_id}`	AI生成の顧客分析・推奨アクション・見積シミュレーション
学習ダッシュボード	`/learning`	スコアリング精度推移・アウトリーチPDCA・CS品質推移。受注/失注パターン分析

---

8. 既存エージェント再利用マッピング

8.1 Layer 2 マイクロエージェント再利用一覧

マイクロエージェント	SFAでの用途	CRMでの用途	CSでの用途
extractor	フォームデータ構造化	要望テキスト分類	チケット分類・緊急度判定
generator	提案書・見積書・契約書PDF生成	月次レポート生成	AI回答ドラフト生成
rule_matcher	リードスコアリング	ヘルススコアアラート判定	チケットルーティング
calculator	見積金額・割引計算	MRR/ARR/NRR算出	SLA残時間計算
message	フォローメール・お礼メール	オンボードシーケンス	サポート回答メール
validator	見積書整合性チェック	売上データ整合性	AI回答品質チェック
saas_reader	リード情報取得	利用データ収集	顧客コンテキスト取得
saas_writer	DB保存・Slack通知	顧客情報更新	チケット保存
ocr	名刺OCR（将来）	—	添付ファイル読取
compliance	契約書法令チェック	—	—
diff	見積バージョン比較	契約変更差分	—
table_parser	—	—	—

8.2 Layer 0 BPO Manager 再利用

コンポーネント	SFA/CRM/CSでの用途
EventListener	フォーム送信、メール着信、Webhook受信を検知してパイプラインをトリガー
ScheduleWatcher	オンボードシーケンス、NPS定期送信、SLAチェック、月次レポート
ConditionEvaluator	リードスコアによる自動/手動振り分け、AI回答の自信度による対応分岐
TaskRouter	パイプライン選択・実行。execution_level（auto/suggest/manual）で承認フロー制御

8.3 Brain モジュール再利用

モジュール	用途
brain/knowledge/qa.py	CSのFAQ自動回答。knowledge_items を検索してサポート回答を生成
brain/knowledge/search.py	類似チケット検索。過去解決策のサジェスト
brain/proactive/analyzer.py	解約リスク検知。拡張提案。次のベストアクション提案
brain/genome/	業種テンプレートから提案書のペインポイント・ソリューションを自動マッピング

---

9. 自動化フロー全体像（イベントドリブン）

```
【インバウンドイベント】                    【自動アクション】

Webフォーム送信 ──→ EventListener ──→ lead_qualification_pipeline
                                        ├→ スコア判定
                                        ├→ DB保存
                                        ├→ Slack通知
                                        └→ スコア≥70: proposal_generation_pipeline へ

提案書開封検知 ──→ EventListener ──→ lead_activities に記録
                                        └→ 3回開封: Slack に「ホットリード」通知

見積承認クリック ──→ EventListener ──→ quotation_contract_pipeline Phase B へ

CloudSign署名完了 ──→ Webhook ──→ contracts.status = "signed"
                                        ├→ customers テーブルに自動作成
                                        ├→ freee 請求書自動作成・送付
                                        └→ customer_lifecycle_pipeline 開始

freee入金確認 ──→ Webhook/ポーリング ──→ revenue_records に自動記録
                                        └→ 月末: 月次レポート自動生成

サポートメール着信 ──→ Webhook ──→ support_auto_response_pipeline
                                        ├→ AI回答（confidence≥0.85）
                                        ├→ 人間レビュー待ち（0.5-0.85）
                                        └→ エスカレーション（<0.5）

NPS調査回答 ──→ EventListener ──→ customer_health 更新
                                        └→ NPS≤6: 即アラート → 個別フォロー

【スケジュールイベント】                    【自動アクション】

毎日 9:00 ──→ ScheduleWatcher ──→ ヘルススコア再計算 → アラート
毎日 10:00 ──→ ScheduleWatcher ──→ SLA違反チェック → エスカレーション
毎週月曜 ──→ ScheduleWatcher ──→ パイプラインサマリー Slack投稿
毎月1日 ──→ ScheduleWatcher ──→ MRR/チャーンレポート生成
毎月15日 ──→ ScheduleWatcher ──→ 要望ランキング更新 → プロダクトチーム共有
```

10. KPI・ダッシュボード設計

10.1 SFA KPI

指標	計算方法	目標
リード→商談 転換率	qualified / total_leads	≥ 30%
商談→受注 転換率	won / total_opportunities	≥ 25%
平均セールスサイクル	avg(won_date - lead_created_at)	≤ 21日
提案書→見積 進行率	quotation_stage / proposal_stage	≥ 60%
AI自動処理率	auto_handled / total_leads	≥ 70%

10.2 CRM KPI

指標	計算方法	目標
MRR	Σ active_customers.mrr	成長率 ≥ 15%/月
NRR (Net Revenue Retention)	(MRR + expansion - contraction - churn) / MRR_prior	≥ 110%
月次チャーン率	churned_mrr / mrr_start	≤ 3%
NPS	promoters% - detractors%	≥ 30
平均ヘルススコア	avg(customers.health_score)	≥ 70

10.3 CS KPI

指標	計算方法	目標
AI自動解決率	ai_resolved / total_tickets	≥ 60%
平均初回応答時間 (FRT)	avg(first_response_at - created_at)	≤ 5分
平均解決時間	avg(resolved_at - created_at)	≤ 4時間
CSAT	avg(satisfaction_score) / 5	≥ 4.2
SLA達成率	on_time / total_tickets	≥ 95%

11. 実装計画

Phase 0（Week 1）：マーケAI・契約AI 吸収統合

```
Task 0-1: マーケAI部品を shachotwo-app に移植
  ├→ research/enricher.py → workers/micro/company_researcher.py
  ├→ signals/detector.py → workers/micro/signal_detector.py
  ├→ scheduling/calendar_api.py → workers/micro/calendar_booker.py
  ├→ research/gbizinfo.py → workers/connector/gbizinfo.py
  ├→ outreach/form_sender.py → workers/connector/playwright_form.py
  ├→ outreach/personalize.py → llm/prompts/outreach_personalize.py
  └→ sheet/ → workers/connector/google_sheets.py

Task 0-2: 契約AI部品を shachotwo-app に移植
  ├→ contract/cloudsign.py → workers/connector/cloudsign.py
  ├→ contract/estimate.py（WeasyPrint） → workers/micro/pdf_generator.py
  └→ templates/ → workers/bpo/sales/templates/

Task 0-3: 移植後の動作確認テスト
Task 0-4: 元リポジトリをアーカイブ（git archive）
```

Phase 1（Week 2-3）：DB + マーケ + SFA基盤

```
Task 1: DBマイグレーション（15テーブル追加: SFA/CRM/CS 11 + 学習 4）
Task 2: routers/ スケルトン全7本（marketing/sales/crm/support/upsell/learning/webhooks）
Task 3: outreach_pipeline.py（パイプライン⓪、マーケAI部品組合せ）
Task 4: lead_qualification_pipeline.py（パイプライン①）
Task 5: EmailConnector（SendGrid）
Task 6: フロントエンド: アウトリーチダッシュボード + リードボード画面
```

Phase 2（Week 4-5）：提案書・見積書・契約書 全自動化

```
Task 7: proposal_generation_pipeline.py（パイプライン②）
Task 8: quotation_contract_pipeline.py（パイプライン③、契約AI部品組合せ）
Task 9: CloudSignConnector動作確認
Task 10: freeeConnector請求書拡張
Task 11: 提案書PDFテンプレート（16業種）
Task 12: フロントエンド: 商談パイプライン + 見積・契約画面
```

Phase 3（Week 6-7）: CRM + CS + アップセル支援

```
Task 13: customer_lifecycle_pipeline.py（パイプライン④）
Task 14: revenue_request_pipeline.py（パイプライン⑤）
Task 15: support_auto_response_pipeline.py（パイプライン⑥）
Task 16: upsell_briefing_pipeline.py（パイプライン⑦）
Task 17: フロントエンド: 顧客360度 + CSチケット + アップセル画面
Task 18: KPIダッシュボード
```

Phase 4（Week 8-9）: 学習フィードバックループ + 統合テスト

```
Task 19: win_loss_feedback_pipeline.py（パイプライン⑧）
Task 20: cs_feedback_pipeline.py（パイプライン⑨）
Task 21: スコアリングモデル自動再学習の仕組み
Task 22: 学習ダッシュボード
Task 23: E2Eテスト（アウトリーチ→リード→契約→CS→アップセルの全フロー）
Task 24: AI精度チューニング + セキュリティレビュー
```

---

12. 実装ファイル構成

```
shachotwo-app/
├── workers/
│   ├── bpo/
│   │   └── sales/                          # ← 新規: 全社自動化パイプライン
│   │       ├── __init__.py
│   │       ├── pipelines/
│   │       │   ├── __init__.py
│   │       │   ├── outreach_pipeline.py               # ⓪ マーケAI吸収
│   │       │   ├── lead_qualification_pipeline.py      # ① リードクオリフィ
│   │       │   ├── proposal_generation_pipeline.py     # ② 提案書自動生成
│   │       │   ├── quotation_contract_pipeline.py      # ③ 見積・契約（契約AI吸収）
│   │       │   ├── customer_lifecycle_pipeline.py      # ④ 顧客ライフサイクル
│   │       │   ├── revenue_request_pipeline.py         # ⑤ 売上・要望管理
│   │       │   ├── support_auto_response_pipeline.py   # ⑥ CS自動対応
│   │       │   ├── upsell_briefing_pipeline.py         # ⑦ アップセル支援
│   │       │   ├── win_loss_feedback_pipeline.py       # ⑧ 受注/失注学習
│   │       │   └── cs_feedback_pipeline.py             # ⑨ CS品質学習
│   │       └── templates/
│   │           ├── proposal_template.html     # 提案書（契約AI由来）
│   │           ├── quotation_template.html    # 見積書（契約AI由来）
│   │           ├── contract_template.html     # 契約書（契約AI由来）
│   │           └── resume_templates/          # LP/レジュメ（マーケAI由来）
│   │               ├── construction.md
│   │               ├── manufacturing.md
│   │               └── dental.md  ... etc
│   │
│   ├── micro/                              # ← 新規マイクロエージェント追加
│   │   ├── company_researcher.py           # マーケAI/research/ 吸収
│   │   ├── signal_detector.py              # マーケAI/signals/ 吸収
│   │   ├── pdf_generator.py                # 契約AI/estimate.py 吸収（WeasyPrint）
│   │   ├── calendar_booker.py              # マーケAI/scheduling/ 吸収
│   │   └── ... (既存12種はそのまま)
│   │
│   └── connector/
│       ├── email.py                         # ← 新規: SendGrid連携
│       ├── cloudsign.py                     # ← 契約AI/cloudsign.py 吸収
│       ├── freee.py                         # ← 既存: 請求書発行メソッド追加
│       ├── gbizinfo.py                      # ← マーケAI/research/gbizinfo.py 吸収
│       ├── google_calendar.py               # ← マーケAI/scheduling/calendar_api.py 吸収
│       ├── google_sheets.py                 # ← マーケAI/sheet/ 吸収
│       ├── playwright_form.py               # ← マーケAI/outreach/form_sender.py 吸収
│       └── intercom.py                      # ← 新規: Intercom連携（オプション）
│
├── routers/
│   ├── marketing.py                         # ← 新規: マーケルーター
│   ├── sales.py                             # ← 新規: SFAルーター
│   ├── crm.py                               # ← 新規: CRMルーター
│   ├── support.py                           # ← 新規: CSルーター
```

```
│   ├── upsell.py                          # ← 新規: アップセル支援ルーター
│   ├── learning.py                        # ← 新規: 学習フィードバックルーター
│   └── webhooks.py                        # ← 新規: Webhook受信ルーター
│
├── llm/
│   └── prompts/
│       ├── outreach_personalize.py        # ← マーケAI/outreach/personalize.py 吸収
│       ├── pain_estimation.py             # ← マーケAI/research/enricher.py 吸収
│       ├── sales_proposal.py              # ← 新規: 提案書生成プロンプト
│       ├── sales_qualification.py         # ← 新規: リード判定プロンプト
│       └── support_response.py            # ← 新規: CS回答生成プロンプト
│
├── db/
│   └── migrations/
│       ├── 020_sfa_crm_cs_tables.sql      # ← 新規: 11テーブル（SFA/CRM/CS）
│       └── 021_learning_tables.sql        # ← 新規: 4テーブル（学習ループ）
│
├── frontend/
│   └── src/app/(authenticated)/
│       ├── marketing/                      # ← 新規: マーケ画面群
│       │   ├── outreach/page.tsx
│       │   ├── research/page.tsx
│       │   └── ab-tests/page.tsx
│       ├── sales/                          # ← 新規: SFA画面群
│       │   ├── leads/page.tsx
│       │   ├── pipeline/page.tsx
│       │   ├── proposals/page.tsx
│       │   ├── contracts/page.tsx
│       │   └── forecast/page.tsx
│       ├── crm/                            # ← 新規: CRM画面群
│       │   ├── customers/page.tsx
│       │   ├── customers/[id]/page.tsx
│       │   ├── revenue/page.tsx
│       │   └── requests/page.tsx
│       ├── support/                        # ← 新規: CS画面群
│       │   ├── tickets/page.tsx
│       │   ├── tickets/[id]/page.tsx
│       │   └── metrics/page.tsx
│       ├── upsell/                         # ← 新規: アップセル画面群
│       │   ├── page.tsx
│       │   └── [customer_id]/page.tsx
│       └── learning/                       # ← 新規: 学習ダッシュボード
│           └── page.tsx
│
└── tests/
    ├── workers/bpo/sales/                  # ← 新規: パイプラインテスト
    ├── workers/micro/                      # ← 新micro-agentテスト追加
    ├── workers/connector/                  # ← 新規: コネクタテスト
    └── routers/                            # ← 新規: ルーターテスト
```

13. セキュリティ考慮事項

項目	対策
見込み客PII	leads/customers の個人情報（氏名・メール・電話）は既存PII検出で保護
メール送信	SendGrid Sender Authentication（SPF/DKIM/DMARC）必須。なりすまし防止
電子署名	CloudSign は電子署名法準拠。署名済みPDFは暗号化保存
請求情報	freee側で管理。口座情報等はfreeeから直接送付
Webhook認証	各サービスの署名検証（SendGrid: Event Webhook Verification, CloudSign: IP制限+署名, freee: OAuth）
社内アクセス制御	管理画面は社内メンバーのみ。Supabase Auth + IP制限（オプション）
AI回答品質	confidence < 0.5 は絶対に自動送信しない。人間レビュー必須
AI送信メールの免責	自動送信メールのフッターに「本メールはAIアシスタントにより送信されています」を明記

---

14. コスト構造（社内ツール）

> これは社外販売する機能ではなく、自社の営業コスト削減のための社内ツール。
> 「営業1人分の人件費 vs AIエージェント運用コスト」で投資判断する。

14.1 運用コスト見積

項目	月額概算	備考
SendGrid	¥0〜¥5,000	月1万通まで無料。提案書・フォロー・CS回答含む
CloudSign	¥10,000〜	月5件〜。契約書電子署名
LLM（Gemini Flash）	¥5,000〜¥20,000	提案書生成・CS回答・リードスコアリング
Supabase	¥0〜¥3,000	既存インフラに相乗り
合計	**¥15,000〜¥38,000/月**	**営業1人の月給の1/10以下**

14.2 ROI試算

```
【Before】人間営業1名 = ¥500,000/月（給与+社保+交通費）
  ・対応可能リード: 30件/月
  ・提案書作成: 2時間/件
  ・稼働時間: 160h/月

【After】AIエージェント = ¥30,000/月
  ・対応可能リード: 無制限（24h365日）
  ・提案書作成: 3分/件（AI自動生成）
  ・人間介入: エスカレーション時のみ（月数時間）

→ 年間削減額: (¥500,000 − ¥30,000) × 12 = ¥5,640,000
→ 初期開発投資の回収: 2–3ヶ月
```

14.3 将来の社外展開可能性

本システムが社内で成果を出した場合、**顧客企業にも同機能を提供**できる。
その場合はシャチョツーの追加モジュールとして以下の料金で提供可能:

モジュール	月額	内容
セールスAI	¥150,000	SFA全自動化 + CRM + CS
セールスAI Lite	¥50,000	CRM + CS のみ

company_idカラムを残しているので、マルチテナント化は容易。
```
```